

UD03: Entrada y salida de información

Contenido

Introducción a la entrada y salida de datos (E/S).....	2
1. Entrada y salida en consola	2
1.1 Entrada desde teclado (input())	2
1.2 Salida a pantalla (print()).....	2
1.3 Formatos de visualización.....	3
2. Ficheros en Python	5
2.1 Tipos de flujo: bytes y caracteres	5
2.2 Clases y funciones relacionadas con E/S de ficheros	5
2.3 Apertura y cierre de ficheros.....	5
2.3.1 Análisis detallado del contexto de gestión with	6
2.4 Modos de acceso a ficheros.....	7
2.5 Escritura y lectura de información	7
2.6 Ficheros de datos y registros	8
2.7 Creación y eliminación de ficheros y directorios	8
2.8 Utilización de los sistemas de ficheros	9
3. Interfaces gráficas de usuario (GUI)	10
3.1 Introducción y herramientas disponibles	10
3.2 Creación de interfaces gráficas simples.....	10
3.3 Concepto de evento	11
3.4 Creación de controladores de eventos.....	11
3.5 Entrada y salida mediante GUI	12
4. Tratamiento de documentos en lenguajes de intercambio de datos.....	13
4.1 Lectura y escritura de ficheros JSON.....	13
4.2 Manipulación de datos XML	14
4.3 Lectura y escritura de ficheros CSV.....	14
4.4 Conversión entre formatos.....	15

Introducción a la entrada y salida de datos (E/S)

La **entrada y salida (E/S)**, o *Input/Output (I/O)* en inglés, se refiere a la comunicación entre el programa y el mundo exterior. Un programa puede recibir datos (entrada) de diferentes fuentes, como el teclado, un archivo, o un sensor, y puede enviar datos (salida) a una pantalla, un archivo, o una impresora. Esta capacidad es fundamental para que las aplicaciones sean interactivas y útiles.

1. Entrada y salida en consola

La forma más básica de E/S en Python es a través de la consola o terminal.

1.1 Entrada desde teclado (input())

La función `input()` se utiliza para obtener datos del usuario a través del teclado. Detiene la ejecución del programa, muestra un mensaje opcional y espera a que el usuario escriba algo y presione *Enter*. El valor que devuelve siempre es una **cadena de texto (str)**.

```
nombre = input("Por favor, introduce tu nombre: ")
```

```
print(f"Hola, {nombre}")
```

```
# El valor introducido, aunque sean números, es una cadena.
```

```
edad_str = input("Introduce tu edad: ")
```

```
print(f"Tipo de la variable 'edad_str': {type(edad_str)}")
```

```
# Es necesario convertirlo a un tipo numérico si se va a operar con él.
```

```
edad = int(edad_str)
```

```
print(f"Tu edad es: {edad}")
```

1.2 Salida a pantalla (print())

La función `print()` es la más utilizada para mostrar información en la consola. Puede mostrar uno o varios valores, separados por comas. Por defecto, los valores se separan con un espacio y, al final, se añade un salto de línea.

Impresión de una cadena

```
print("¡Hola mundo!")
```

Impresión de múltiples valores (se separan por un espacio por defecto)

```
nombre = "Ana"
```

```
edad = 25
```

```
print("Nombre:", nombre, "Edad:", edad)
```

Cambio del separador y del final de línea

```
print("Valor1", "Valor2", "Valor3", sep=" - ", end=".\\n")
```

Salida: Valor1 - Valor2 - Valor3.

1.3 Formatos de visualización

Para crear salidas más estructuradas y legibles, Python ofrece varias formas de formatear cadenas.

- **Formato con la función `format()`:** Permite insertar valores en una cadena usando marcadores de posición (`{}`).

```
nombre = "Pepe"
```

```
edad = 30
```

```
print("Mi nombre es {} y tengo {} años.".format(nombre, edad))
```

También se puede usar por posición

```
print("Mi nombre es {1} y tengo {0} años.".format(edad, nombre))
```

- **Cadenas con formato f-string (la más moderna y recomendada):** Se usa una f antes de la cadena y se insertan las variables directamente entre llaves. Es la forma más legible y concisa.

```
nombre = "Laura"
```

```
edad = 28
```

```
print(f"Mi nombre es {nombre} y tengo {edad} años.")
```

```
# Permite expresiones dentro de las llaves
```

```
print(f"Dentro de 5 años, {nombre} tendrá {edad + 5} años.")
```

2. Ficheros en Python

Trabajar con ficheros te permite almacenar datos de forma persistente. La E/S de ficheros es un concepto central en el desarrollo de aplicaciones.

2.1 Tipos de flujo: bytes y caracteres

- **Ficheros de texto (caracteres):** Se leen y escriben como secuencias de caracteres, codificados (por ejemplo, en UTF-8). Es la forma más habitual para ficheros de texto plano (.txt, .csv, etc.).
- **Ficheros binarios (bytes):** Se leen y escriben como secuencias de bytes. Son ideales para trabajar con imágenes, videos, ejecutables, y otros tipos de datos no textuales.

2.2 Clases y funciones relacionadas con E/S de ficheros

La función principal es **open()**, que devuelve un objeto de tipo fichero que se puede usar para leer, escribir o añadir contenido.

2.3 Apertura y cierre de ficheros

Un fichero debe ser abierto antes de ser usado y cerrado después para liberar los recursos del sistema. Esto evita problemas como la corrupción de datos o el agotamiento de recursos.

El método `close()` se encarga de cerrar un fichero, pero la forma más segura de trabajar es usando la sentencia `with`, que garantiza que el fichero se cierre automáticamente, incluso si ocurre un error.

Ejemplo 1 – Método manual (propenso a errores):

```
fichero = open("archivo.txt", "w")  
  
fichero.write("Contenido de prueba.")  
  
fichero.close()
```

Ejemplo 2 – Método recomendado (usando with):

```
with open("archivo.txt", "w") as f:  
    f.write("Otro contenido")  
  
# Al salir del bloque, el archivo se cierra automáticamente
```

2.3.1 Análisis detallado del contexto de gestión with

El bloque with permite garantizar la **liberación de recursos de manera segura y automática**. Para entenderlo, conviene desglosar dos conceptos clave:

1. La declaración with establece un contexto de gestión

- El with crea un bloque de código especial, llamado **contexto de gestión**, donde se utiliza un recurso (archivo, conexión, lock, etc.) de forma controlada.
- Al salir del bloque, Python ejecuta automáticamente la acción de limpieza asociada al recurso, incluso si ocurre un error, evitando que se quede abierto o bloqueado.
- En términos técnicos, el bloque with llama a los métodos especiales `__enter__()` al inicio y `__exit__()` al final del contexto.

2. `open()` devuelve un objeto de archivo que actúa como gestor de contexto

- El objeto devuelto por `open()` implementa los métodos `__enter__` y `__exit__`, por lo que puede integrarse directamente con with.
- Esto significa que sabe cómo abrir el recurso al inicio del bloque y cerrarlo al salir, asegurando que la liberación de recursos sea automática y segura.

Ejemplo práctico

with open("archivo.txt", "w") as f: # open() devuelve un gestor de contexto

 f.write("Hola mundo") # Se utiliza el recurso dentro del contexto

Al salir del bloque, `__exit__` se llama automáticamente y el archivo se cierra

Análisis paso a paso del ejemplo

1. `open("archivo.txt", "w")`

- Devuelve un objeto gestor de contexto que sabe cómo abrir y cerrar el archivo.

2. `with ... as f:`

- Establece el contexto de gestión, asegurando que el recurso se libere al finalizar el bloque.
- `f` es la variable que referencia el objeto de archivo dentro del bloque.

3. **f.write("Hola mundo")**

- Realiza la operación dentro del contexto seguro.

4. **Al salir del bloque with**

- Python llama automáticamente al método `__exit__()` del gestor de contexto.
- El archivo se cierra sin necesidad de llamar a `f.close()`.

2.4 Modos de acceso a ficheros

El segundo argumento de `open()` es el modo de apertura:

- 'r' (lectura): el modo por defecto. Si el fichero no existe, da un error.
- 'w' (escritura): crea un fichero si no existe o sobrescribe uno existente.
- 'a' (adjuntar/añadir): abre el fichero para escribir, añadiendo el contenido al final. Si no existe, lo crea.
- 'b' (binario): se combina con los anteriores ('rb', 'wb') para trabajar con ficheros binarios.
- 'r+' (lectura y escritura): abre el fichero para leer y escribir.

2.5 Escritura y lectura de información

Este apartado explica los métodos más utilizados para operaciones de escritura y lectura, así como ejemplos prácticos de su uso:

- **Escritura (`write()` y `writelines()`):**
 - `write(cadena)`: escribe una cadena en el fichero.
 - `writelines(lista_de_cadenas)`: escribe una lista de cadenas. No añade saltos de línea automáticamente.
- **Lectura (`read()`, `readline()` y `readlines()`):**
 - `read()`: lee todo el contenido del fichero como una sola cadena.
 - `readline()`: lee la siguiente línea del fichero, incluyendo el salto de línea.
 - `readlines()`: lee todas las líneas y las devuelve como una lista de cadenas.

Escritura

with open("ejemplo.txt", "w") as f:

 f.write("Línea 1\n")

 f.write("Línea 2\n")

Lectura

with open("ejemplo.txt", "r") as f:

 contenido = f.read()

 print("Contenido completo:\n", contenido)

Lectura línea a línea (forma recomendada para grandes ficheros)

with open("ejemplo.txt", "r") as f:

 for linea in f:

 print("Línea leída:", linea.strip()) # .strip() elimina el salto de línea

2.6 Ficheros de datos y registros

En la programación profesional, los ficheros no solo se usan para texto plano, sino también para almacenar datos estructurados. Python puede trabajar con formatos como CSV (usando el módulo csv) o JSON (usando el módulo json), que son ideales para guardar registros.

2.7 Creación y eliminación de ficheros y directorios

El módulo os de Python ofrece funciones para interactuar con el sistema de ficheros:

- os.remove("fichero.txt"): elimina un fichero.
- os.rename("antiguo.txt", "nuevo.txt"): renombra un fichero.
- os.mkdir("nuevo_directorio"): crea un directorio.
- os.rmdir("directorio_vacio"): elimina un directorio (debe estar vacío).

2.8 Utilización de los sistemas de ficheros

Además de `os`, el módulo `pathlib` (más moderno y recomendado) proporciona una forma orientada a objetos para manejar rutas y ficheros, facilitando la creación de código multiplataforma y legible.

3. Interfaces gráficas de usuario (GUI)

Una GUI (*Graphical User Interface*) permite a los usuarios interactuar con un programa a través de elementos gráficos como botones, cuadros de texto y ventanas, en lugar de la consola.

3.1 Introducción y herramientas disponibles

Python tiene varias bibliotecas para crear GUIs:

- **Tkinter:** Es la biblioteca estándar de Python. Es simple y viene preinstalada, ideal para proyectos sencillos y para aprender los conceptos básicos.
- **PyQt / PySide:** Ofrecen herramientas muy potentes y profesionales para crear interfaces complejas y de gran calidad.
- **Kivy:** Orientada a la creación de aplicaciones multiplataforma, ideal para desarrollo móvil.
- **wxPython:** Otra biblioteca popular y potente.

3.2 Creación de interfaces gráficas simples

El proceso básico con Tkinter es:

1. Crear la ventana principal (`Tk()`).
2. Crear los *widgets* (botones, etiquetas, etc.).
3. Posicionar los *widgets* en la ventana.
4. Iniciar el bucle principal (`mainloop()`) para que la ventana se muestre y escuche eventos.

```
import tkinter as tk
```

```
# 1. Crear la ventana principal
```

```
ventana = tk.Tk()
```

```
ventana.title("Mi primera GUI")
```

```
ventana.geometry("300x200") # Ancho x Alto
```

2. Crear un widget (etiqueta)

```
etiqueta = tk.Label(ventana, text="¡Hola, mundo GUI!")
```

```
etiqueta.pack(pady=20) # 3. Posicionar el widget
```

4. Iniciar el bucle principal de eventos

```
ventana.mainloop()
```

3.3 Concepto de evento

Un **evento** es una acción que ocurre durante la ejecución de un programa. Por ejemplo:

- Un clic del ratón.
- Pulsar una tecla.
- Cambiar el tamaño de una ventana.
- Mover el puntero del ratón. Las GUIs están orientadas a eventos; el programa no ejecuta código secuencialmente, sino que reacciona a lo que el usuario hace.

3.4 Creación de controladores de eventos

Un **controlador de eventos** es una función que se ejecuta cuando ocurre un evento específico. Por ejemplo, una función que se llama cuando el usuario hace clic en un botón.

Python

```
import tkinter as tk
```

```
def saludar():
```

```
    """Función que se ejecuta al hacer clic en el botón."""
```

```
    etiqueta.config(text="¡Botón pulsado!")
```

```
ventana = tk.Tk()
```

```
ventana.title("Controlador de Eventos")
```

```
etiqueta = tk.Label(ventana, text="Presiona el botón...")
```

```
etiqueta.pack(pady=10)
```

```
# Crear un botón y asociarle la función 'saludar' con el parámetro 'command'
```

```
boton = tk.Button(ventana, text="Pulsa aquí", command=saludar)
```

```
boton.pack(pady=10)
```

```
ventana.mainloop()
```

3.5 Entrada y salida mediante GUI

La entrada y salida ya no se hacen en la consola, sino a través de *widgets* específicos:

- **Entrada:** Se usan widgets como Entry (cuadro de texto de una línea) o Text (área de texto multilinea) para que el usuario introduzca datos.
- **Salida:** Se usa un widget Label (etiqueta) para mostrar texto, o se pueden actualizar los widgets de entrada para mostrar resultados.

4. Tratamiento de documentos en lenguajes de intercambio de datos

JSON, XML y CSV son estándar para el intercambio de datos entre aplicaciones.

4.1 Lectura y escritura de ficheros JSON

El módulo json de Python facilita la conversión entre datos Python y cadenas JSON.

- **json.dumps(obj):** Convierte un objeto Python a una cadena JSON.
- **json.loads(str):** Convierte una cadena JSON a un objeto Python.
- **json.dump(obj, f):** Escribe un objeto Python directamente en un fichero.
- **json.load(f):** Lee un fichero JSON y lo convierte a un objeto Python.

```
import json
```

```
# Escribir en un fichero JSON
```

```
datos = {"nombre": "Carlos", "edad": 25}
```

```
with open("datos.json", "w") as f:
```

```
    json.dump(datos, f, indent=4)
```

```
# Leer desde un fichero JSON
```

```
with open("datos.json", "r") as f:
```

```
    datos_leidos = json.load(f)
```

```
    print(datos_leidos["nombre"])
```

```
# Salida: Carlos
```

4.2 Manipulación de datos XML

Para trabajar con XML, el módulo **xml.etree.ElementTree** es la herramienta estándar que viene con Python. Permite parsear (analizar), crear y modificar documentos XML de manera sencilla. Librerías como lxml ofrecen más funcionalidades y mejor rendimiento, pero ElementTree es suficiente para la mayoría de los casos.

```
import xml.etree.ElementTree as ET

# Crear un documento XML

raiz = ET.Element("persona")

nombre = ET.SubElement(raiz, "nombre")

nombre.text = "Ana"

edad = ET.SubElement(raiz, "edad")

edad.text = "28"

# Guardar en un fichero

arbol = ET.ElementTree(raiz)

arbol.write("persona.xml")
```

4.3 Lectura y escritura de ficheros CSV

El formato **CSV** (Comma Separated Values) se utiliza para almacenar datos tabulares. Python ofrece el módulo csv para manejar estos ficheros de forma sencilla.

- `csv.reader(f)` → Lee filas desde un fichero CSV.
- `csv.writer(f)` → Escribe filas en un fichero CSV.
- `writerow(lista)` → Escribe una fila.
- `DictReader` y `DictWriter` → Permiten trabajar con diccionarios.

Ejemplo:

```
import csv
```

```
# Escribir un fichero CSV
```

```
with open("alumnos.csv", "w", newline="", encoding="utf-8") as f:
```

```
    escritor = csv.writer(f)
```

```
    escritor.writerow(["nombre", "nota"])
```

```
    escritor.writerow(["Ana", 8.5])
```

```
    escritor.writerow(["Jorge", 6.0])
```

```
    escritor.writerow(["Marta", 9.2])
```

```
# Leer un fichero CSV
```

```
with open("alumnos.csv", "r", encoding="utf-8") as f:
```

```
    lector = csv.reader(f)
```

```
    next(lector) # Saltar cabecera
```

```
    for nombre, nota in lector:
```

```
        print(f"{nombre}: {nota}")
```

4.4 Conversión entre formatos

En el desarrollo es habitual convertir datos entre formatos:

- Diccionario → JSON → enviar a una API
- XML → Diccionario → procesar en el programa
- CSV → Diccionario o JSON → análisis de datos

Python facilita estas conversiones gracias a la similitud estructural entre **diccionarios**, **JSON**, **XML** y **tablas CSV**.

Ejemplo: convertir CSV a diccionario Python

import csv

with open("alumnos.csv", "r", encoding="utf-8") as f:

lector = csv.DictReader(f)

lista_alumnos = [fila for fila in lector]

print(lista_alumnos)

Salida: [{'nombre': 'Ana', 'nota': '8.5'}, {'nombre': 'Jorge', 'nota': '6.0'}, ...]